

Why the Compiled 4D Prefix Implementation Is Faster

Implementation audit and reproducible 4D benchmarks

4 October 2026

Abstract

The compiled prefix implementation in `HV4DMagnitude` is faster than the new numerical Chan backend on the 4D inputs tested here. On spherical fronts with 20 or 40 points it is 1.59–2.11 times faster than the existing Python Chan $d/3$ implementation. At 20 points it is 84.59–96.99 times faster than the new default Numba backend. Two controlled comparisons explain this: the old implementation compiles the entire numerical $2 + 2$ loop, while the new compression policy creates costly intermediate term states whose manipulation largely remains in Python. This report separates empirical speed, correctness, and worst-case complexity. No solver was modified, and only 4D was tested.

1 Question and measured answer

The earlier comparison of numerical Chan implementations omitted the compiled prefix solver in `HV4DMagnitude`. It therefore did not assess the user’s earlier Numba approach. We now compare that solver [1] with the new numerical engine and the two existing Python Chan implementations [2, 3]. The user’s recollection of a faster compiled 4D implementation is confirmed.

Family seed / n	Old JIT	Chan $d/3$	New JIT	No-comp.	Chan $d/2$
Sphere 42 / 10	5.295	7.519	9.259	10.972	1.515
Sphere 42 / 20	21.521	35.635	2087.270	45.190	5.167
Sphere 42 / 40	62.343	111.593	budget	142.064	19.298
Sphere 99 / 20	21.924	34.805	1854.674	37.848	5.983
Sphere 99 / 40	52.861	111.451	budget	157.691	21.045
Grid 7 / 20	0.834	2.225	5.549	5.907	0.323
Uniform 17 / 20	14.546	18.634	832.086	29.710	2.405

Table 1: Median of three warm end-to-end calls, in milliseconds. “budget” denotes a 35-second whole-worker timeout, not a per-call lower bound.

Old JIT uses `PrefixHV4D` with `base_hard=2` and `fast=True`.

New JIT uses `NumericalChanNumba` with dimension four and the same base threshold, the shared backend used by the new 4D wrapper. No-comp. is that same new backend

with `block_levels=10**6`; this diagnostic avoids compression on these inputs but has no claimed worst-case guarantee. Chan $d/3$ and $d/2$ use no Numba.

Chan $d/2$ is fastest on all seven measured inputs. On the four spherical $n = 20, 40$ cases, it is 2.51–4.16 times faster than old JIT. These are small input sizes: this observation neither contradicts an asymptotic advantage for a different algorithm nor establishes an eventual crossover. No exponent is fitted to this experiment.

2 Fair inputs, timing, and validation

All algorithms compute the same ordinary four-dimensional hypervolume. Old prefix and both Chan references receive nonnegative minimization points p with reference r . The new engine receives the reflected anchored corners $r - p$. Passing $-p$ with zero reference to the old prefix solver would be unsafe: its grids are anchored at zero and discard negative cuts. The conversion for the new engine is included in its timed call. No implementation uses a dominance prefilter, and the shared base threshold is two hard boxes wherever that option exists.

Each algorithm and dataset uses a fresh process. One initial call precedes three timed calls; the table reports their median. Imports and the initial call are excluded, but packing, preprocessing, and complete evaluation are included. The initial call may load a Numba disk cache; its recorded duration is not a pure compilation time. Workers run sequentially in a reproducibly shuffled order within each dataset. There is no Numba parallelism or fastmath. A timeout includes process startup, imports, first use and all three repeats. It gives no measured per-call ratio for an unfinished row.

Sphere inputs are absolute independent Gaussian vectors normalized to unit Euclidean length, with $r = (1.2, 1.2, 1.2, 1.2)$. Seed 42 consumes sizes 5, 10, 20, 40 successively, discarding the five-point case; seed 99 consumes 20, 40. Grid inputs use independent integers from zero through three divided by four; uniform inputs use independent coordinates in $[0, 1)$. Both have $r = (1, 1, 1, 1)$. Every input is saved, rather than reconstructed from timings. The historical audit did not archive its exact timing inputs: this experiment uses its committed test family, without claiming to reproduce its historical table byte for byte.

All 52 completed rows agree numerically. Against the separate Chan $d/2$ evaluator, maximum scaled disagreement is 7.89×10^{-16} , using $|x - y| / \max(1, |y|)$. The ten-point sphere also passes an independent inclusion–exclusion oracle evaluated with exact rational representations of the supplied binary floating-point inputs; its maximum scaled error is 2.22×10^{-16} . Four new default runs time out: Python and Numba on each of the two forty-point spheres. These checks are empirical validation, not a general numerical-error theorem. Every completed old-JIT row records a Numba nopython signature.

The machine is an AMD Ryzen 7 5800H with eight physical cores, sixteen logical processors, and approximately 16 GiB RAM. Software is Windows 11, CPython 3.12.14, NumPy 2.5.3 and Numba 0.68.0. The measurements do not use confidence intervals, processor pinning, or a controlled dedicated server; three repeats support the large observed differences, not fine percentage claims.

3 The compilation boundary matters

The old implementation packs six staircases, reversed values, target grids, and cumulative source records into numeric arrays. Its `fastkernel.k_prefix` compiles the entire nested target-plane scan: binary searches, clipping source bounds, choosing the source-area formula, and numerical accumulation all execute inside the compiled call.

Disabling JIT *for that same flat kernel* is the cleanest compilation comparison. Table 2 also includes the old object-based evaluator, which is a distinct code path selected by `fast=False`.

Family seed / n	Old flat Python	Old object Python	Flat/JIT	New Python
Sphere 42 / 10	69.091	69.840	13.05	8.843
Sphere 42 / 20	255.417	250.038	11.87	1291.028
Sphere 42 / 40	804.383	785.652	12.90	budget
Sphere 99 / 20	339.409	330.800	15.48	1147.316
Sphere 99 / 40	695.438	715.985	13.16	budget
Grid 7 / 20	1.596	2.260	1.91	3.424
Uniform 17 / 20	139.630	129.447	9.60	500.031

Table 2: Warm medians in milliseconds, except the dimensionless Flat/JIT ratio. “budget” has the same meaning as in Table 1.

On the spherical $n = 20, 40$ cases, compiling the old flat path gives an 11.87–15.48-fold speedup. The new backend instead compiles small helpers for searches, prefix sums, normalization, masks and related array operations. Winner enumeration and much of the copying, comparison and merging of terms remain in Python. Numerous calls on short arrays do not remove that work. “New Python” in Table 2 means the shared numerical engine at dimension four, not the separate specialized `numerical_chan4.py` file. All these timing inputs use floating-point arithmetic, including this otherwise exact-capable Python engine.

4 Compression changes the later cost of evaluation

The new default schedule compresses after a block of four cyclic levels at these input sizes. In 4D this temporarily creates eight variables and eliminates four, producing signed terms. The old prefix solver performs no such compression. The existing Python Chan $d/3$ code additionally gates compression on representation size; it performs none on the seed-42 twenty-point case.

For that case the two new Numba configurations have the same 61 recursive calls and 30 cuts, but very different term work:

Statistic	Default	No compression
Compressions	10	0
Raw emitted terms	10,809	709
Term eliminations	7,441	526
Peak intermediate terms	625	11
Median milliseconds	2,087.270	45.190

The no-compression variant is 46.19 times faster on this case and 49.00 times faster on the other twenty-point sphere. This measures the compression policy *and the states it leaves for later work*, not merely the cost of calling the compression routine. The ten-point sphere and tied-grid case perform no compression under either policy, so their small timing differences are not evidence of a compression effect.

Separate profiling of the seed-42 twenty-point case supports this explanation. The old JIT path records 40,833 Python-visible calls; the new default records 3,285,965, and the new no-compression variant 82,860. Calls internal to compiled code are not counted individually, so these figures describe interpreter overhead, not arithmetic-operation complexity. In the new default profile, about 15.3% of time is cumulative within `push_blocks`, while 78.7% is in base-case evaluation. Expensive states persist beyond compression itself. Prominent Python-side work includes `emitted`, `impose`, `add_edge`, unary multiplication and merging. Profiled durations are never substituted for the unprofiled benchmark medians.

The old solver also absorbs slabs by shrinking geometric cells and reclassifying the remaining orthants; the new engine zeros unary entries without shrinking the cell. This is a further difference, but its individual speed contribution was not isolated here. Cross-implementation counters are not universally comparable: the new node count includes axis-skip calls and its term high-water mark includes temporary elimination outputs.

5 What the numerical $2 + 2$ loop provides

The six pairwise staircases encode constraints between coordinates. After fixing a target cell (u, v) in coordinates 3 and 4, cross constraints give caps $A(u, v), B(u, v)$ on coordinates 1 and 2. The remaining source-plane area $H_{12}(A, B)$ is obtained from the stored records for the first staircase. The complete computation has the numerical form

$$K = \sum_{u,v} w_3(u)w_4(v)\mathbf{1}_{\{v < f_{34}(u)\}} H_{12}(A(u, v), B(u, v)).$$

Here u, v denote representatives of target cells and the w_i are their widths for ordinary volume. The source evaluator uses a two-branch prefix formula; no symbolic term is constructed for each target cell. In the seed-42 twenty-point run there are 81 box evaluations, each making sixteen signed prefix queries, for 1,296 queries in total.

This is a useful compact numerical implementation, but it explicitly visits target-grid cells at every prefix query. It also retains live orthants in descendants and does not implement the new compression schedule. **This audit does not establish an overall $O(n^{4/3} \text{polylog } n)$ bound for that older implementation.** Its own repository marks the overall complexity claim as unverified. The fast constant factors observed here do not supply the missing bound on global cell visits. Conversely, disabling the new engine’s compression is only an ablation: its speed does not preserve a guarantee proved for a different schedule.

6 Magnitude: a separate correctness finding

The old public hypervolume solver hardcodes Lebesgue measure, so the main benchmark concerns ordinary HV. Direct calls to its packed kernel with the magnitude product measure

reveal a separate atom-handling defect. Let all six staircases be constant two, with every axis grid equal to $[0, 1]$. The magnitude of the four-dimensional unit box should be $(1 + 1/2)^4 = 81/16 = 5.0625$. The decomposed and brute evaluators return this value, but the packed kernel returns 2.25. At query $(1, 1, 0, 0)$, expected magnitude is $9/4$, while the packed kernel returns zero. Lebesgue versions of these two checks return the correct values, one and zero.

The target-grid loop substitutes the origin atom for the first interval instead of accounting for both pieces. This defect does not explain the ordinary-HV speed difference. The ordinary-HV route already supports magnitude through the affine identity, for nonnegative anchored generators,

$$\text{Mag}(D(A)) = \text{HV}_4(\{(1 + a_1/2, \dots, 1 + a_4/2) : a \in A\}).$$

To see it, each nonempty intersection of anchored boxes has coordinatewise minimum b and magnitude $\prod_i (1 + b_i/2)$. Its transformed ordinary volume is the same product, since the affine coordinate transformation commutes with minimum. Inclusion–exclusion proves equality for the union. This is an equivalence of numerical set values; it is not a translation of the anchored atomic measure.

Three exact inclusion–exclusion spot checks of this route pass through the old public HV solver: the unit box gives $81/16$, the degenerate box with corner $(1, 1, 0, 0)$ gives $9/4$, and the four-generator set

$$\{(1, 4, 3, 2), (2, 3, 4, 1), (3, 2, 1, 4), (4, 1, 2, 3)\}$$

gives $765/16$. The direct packed-kernel defect is recorded in the accompanying reproducer; no solver source was repaired or changed for these experiments.

7 Reproduction and implications for implementation

The accompanying directory contains the benchmark script, all seven input sets, every raw repeat, first-call durations, counters, profiles, hashes, derived ratios, and the magnitude checks. Sources are pinned to the commits in the bibliography; byte hashes and hashes with normalized line endings are recorded in `provenance.json`. The measured new sources equal the packaged sources apart from line endings.

From the FastHVChan root, after installing the requirements in `NumericalChanHVND` and checking out the pinned old repository at `../HV4DMagnitude`, run the following single command (line breaks are shown for readability):

```
python NumericalChanHV4D/benchmarks/prefix4/compare_prefix4.py
-old ../HV4DMagnitude -new NumericalChanHVND
-reference . -out prefix4-reproduction -profiles
```

The separate `check_prefix4_magnitude.py` accepts `-old ../HV4DMagnitude -out magnitude-checks.json`.

The old compiled $2 + 2$ evaluator is the appropriate practical 4D baseline. A faster new backend should keep a whole contraction loop inside compiled code and control term-state growth. The compression ablation motivates an adaptive policy, but adopting one

while retaining a proved worst-case bound requires additional analysis. These are concrete follow-up targets, not improvements implemented or new complexity theorems established by this report.

References

- [1] M. Emmerich, *HV4DMagnitude*, source snapshot `0c668e7`, inspected 4 October 2026. [Pinned repository](#). Relevant files: `solver.py`, `prefix4.py`, `fastkernel.py`, and `notes/audit.md`.
- [2] M. Emmerich, *FastHVChan*, reference source snapshot `08539aa`, inspected 4 October 2026. [Pinned repository](#). The measured Python files are `chan_orthant_dby3.py` and `chan_hypervolume.py`. These are repository implementations of the Chan approaches, not a claim to benchmark code authored by Chan.
- [3] *NumericalChanHV4D and NumericalChanHVND*, FastHVChan commit `97012f7`. [Pinned shared numerical implementation](#). The 4D Numba wrapper selects dimension four in that shared engine.